

크롤링 원본 CSV → 전처리 → 라벨링 → 임베딩 →

데이터 규모	DBR 11,273건 + HBR 2,062건 = 총 13,335건 (+ NAVER 55,465건 모델 학습용)
처리 단계	① 전처리 → ② 라벨링 → ③ 임베딩+FAISS → ④ 리스크 모델 → ⑤ 클러스터링
핵심 기술	Kiwi 형태소분석, SBERT(ko-sroberta), FAISS, MLP, UMAP, K-means
최종 목적	사용자 전략 입력 → 유사 사례 검색 → 실패 확률(리스크 스코어) 산출

목 차

0. 전체 파이프라인 개요 — 왜 이 과정이 필요한가?

1. STEP 1 전처리 (preprocess.py)

- 1-1 원본 데이터 문제점
- 1-2 텍스트 정제 과정
- 1-3 형태소 분석 (KiwI)
- 1-4 산출물

2. STEP 2 라벨링 (label.py)

- 2-1 왜 자동 라벨링이 필요한가?
- 2-2 Stage 1: 키워드 규칙 기반
- 2-3 Stage 2: SBERT 센트로이드 재분류
- 2-4 라벨링 결과

3. STEP 3 임베딩 + FAISS (embed.py)

- 3-1 임베딩이란?
- 3-2 Sentence-BERT 모델
- 3-3 FAISS 인덱스 구축

4. STEP 4 리스크 스코어링 모델 (risk_model.py)

- 4-1 머신러닝 분류 문제로 정의
- 4-2 모델 비교 및 선택
- 4-3 3-factor 리스크 스코어 공식

5. STEP 5 UMAP + K-means (umap_cluster.py)

- 5-1 UMAP 차원 축소
- 5-2 K-means 클러스터링
- 5-3 시맨틱 맵 활용

6. HBS 데이터 처리 시 주의사항

7. 파이프라인 실행 순서 정리

0. 전체 파이프라인 개요

동아줄 서비스는 '이 전략을 실행하면 실패할 확률이 얼마나 되는가?'를 AI로 진단한다. 그러려면 DBR·HBR의 수만 건 아티클에서 성공/실패 패턴을 기계가 학습할 수 있는 형태로 변환해야 한다. 이 변환 과정이 NLP 파이프라인이다.

**왜
필요한가?**

원본 CSV는 '사람이 읽는 텍스트'다. 컴퓨터(ML 모델)는 숫자만 이해한다. 텍스트 → 숫자 벡터 → 학습 가능한 형태로 만들기 위해 5단계 파이프라인이 필요하다.

전체 흐름도



1-1 원본 데이터 문제점

왜 필요한가? 크롤링한 텍스트에는 HTML 태그, URL, '[사진]', 기자명, 특수문자 등 의미없는 노이즈가 섞여 있다. 그대로 모델에 넣으면 ' ', '"', 'https://' 같은 문자가 단어로 인식되어 학습 품질이 크게 떨어진다. 또한 한국어는 '삼성전자가' '성장하였다' 처럼 조사·어미가 붙어있어 그냥 띄어쓰기로 자르면 같은 단어를 다른 단어로 인식한다.

1-2 텍스트 정제 과정 (clean_text 함수)

제거 대상	예시	처리 방법
URL / 이메일	https://dbr.donga.com/article/...	정규식으로 공백 치환
HTML 태그·엔티티	<p>, , &	정규식으로 제거
캡션·출처 표기	[사진=동아일보], (자료: 삼성)	패턴 매칭 후 제거
기자명 패턴	홍길동 기자, 이영희 특파원	정규식으로 제거
특수문자·기호	★, ©, %, ₩ 등	한/영/숫자/공백만 남김
연속 공백	단어 사이 공백	단일 공백으로 정규화

1-3 형태소 분석 (Kiwiparser)

형태소란 의미를 가진 최소 언어 단위다. '삼성전자가'를 '삼성전자' + '가(조사)'로 분리하는 과정이 형태소 분석이다. 조사·어미 같은 문법 요소를 제거하고 의미 있는 품사만 남긴다.

품사 태그	의미	예시
NNG	일반명사	전략, 시장, 성과, 혁신
NNP	고유명사	삼성, 애플, 동아줄, DBR
VV	동사 (사전형으로 복원)	성장하다, 달성하다, 도산하다
VA	형용사 (사전형으로 복원)	탁월하다, 취약하다
SL	외국어 (영문)	AI, IPO, ROI
XR	어근	급성장, 부도 등 복합명사 어근

예시 '삼성전자가 해외시장에서 급성장하며 영업흑자를 달성하였다.' → ['삼성전자', '해외시장', '급성장', '성장하다', '영업흑자', '달성하다']

1-4 산출물 (DBR_preprocessed.parquet / HBR_preprocessed.parquet)

추가 컬럼	내용	다음 단계 활용
clean_text	노이즈 제거된 평문 텍스트	SBERT 임베딩 입력
tokens	토큰 리스트 (Python list)	통계·시각화
token_str	토큰을 공백으로 이은 문자열	TF-IDF · 키워드 라벨링

n_tokens	토큰 수 (10개 미만 문서는 탈락)	품질 필터
----------	----------------------	-------

2-1 왜 자동 라벨링이 필요한가?

왜 필요한가?	ML 모델이 '성공 패턴'과 '실패 패턴'을 학습하려면 각 아티클에 정답 라벨이 붙어있어야 한다. 13,335건을 사람이 일일이 읽고 라벨을 붙이는 건 불가능하다. 따라서 키워드 규칙 + AI 임베딩 유사도를 이용해 자동으로 라벨을 부여한다.
----------------	---

2-2 Stage 1: 키워드 규칙 기반 스코어링

각 아티클의 token_str에서 미리 정의한 성공/실패 키워드가 몇 개나 등장하는지 센다.

키워드 유형	예시 (일부)
성공 키워드 (~60개)	성공, 성장하다, 흑자, 도약, 혁신, IPO, 달성하다, 선도하다 ...
실패 키워드 (~60개)	실패, 파산, 적자, 구조조정, 도산하다, 몰락, 영업손실 ...

판정 규칙:

조건	결과
성공히트 / (성공+실패히트) $\geq$ 0.65 AND 총 히트 $\geq$ 3	success (성공)
성공히트 / (성공+실패히트) $\leq$ 0.35 AND 총 히트 $\geq$ 3	failure (실패)
그 외 (히트 부족 또는 비율 애매)	neutral (모호 → Stage 2로 넘김)

2-3 Stage 2: SBERT 센트로이드 재분류

왜 필요한가?	키워드 규칙은 단어를 '있다/없다'로만 본다. '실패를 극복한 성공 사례'처럼 성공·실패 키워드가 섞인 애매한 문서는 neutral로 남는다. SBERT 임베딩은 문장의 의미를 벡터로 표현하므로, 의미적으로 성공 글과 더 유사한지 실패 글과 더 유사한지를 판단할 수 있다.
----------------	--

과정:

1단계	Stage 1에서 확정된 success / failure 문서들의 SBERT 임베딩을 각각 평균내서 '센트로이드(중심점)' 벡터를 만든다.
2단계	neutral(모호) 문서의 임베딩과 두 센트로이드의 코사인 유사도를 계산한다.
3단계	유사도가 임계값(0.20) 이상이고, 성공 센트로이드에 더 가까우면 success, 실패 센트로이드에 더 가까우면 failure로 재분류한다.
4단계	임계값 미만이면 의미적으로도 불명확한 글이므로 neutral로 최종 유지한다.

2-4 라벨링 결과

소스	총 건수	success	failure	neutral
DBR	11,273	9,800 (86.9%)	532 (4.7%)	941 (8.3%)

HBR	2,062	2,004 (97.2%)	15 (0.7%)	43 (2.1%)
NAVER	55,465	36,556 (65.7%)	3,328 (6.0%)	15,581 (28.3%)

주의

실패 사례가 전체의 4-6%밖에 없다. 이 클래스 불균형 문제 때문에 모델 학습 시 `class_weight='balanced'`를 반드시 적용해야 한다. 그렇지 않으면 모델이 항상 '성공'이라고 예측해도 정확도 95%가 나오는 척 속인다.

3-1 임베딩이란?

왜 필요한가? 컴퓨터는 텍스트를 직접 비교하지 못한다. 텍스트를 고정 길이의 숫자 벡터로 변환하면 두 문장이 의미적으로 얼마나 비슷한지를 수치(코사인 유사도)로 계산할 수 있다. 이 숫자 벡터가 바로 '임베딩'이다.

직관적 예시 '삼성이 반도체 시장에서 점유율 1위를 달성했다' 와 'SK하이닉스가 메모리 분야 매출 최고 기록을 세웠다'는 단어는 다르지만 의미는 비슷하다. 임베딩 벡터를 계산하면 두 문장의 벡터 거리가 가깝게 나온다.

3-2 Sentence-BERT 모델 (jhgan/ko-sroberta-multitask)

일반 Word2Vec/TF-IDF는 단어 단위 임베딩이라 문맥을 파악하지 못한다. Sentence-BERT는 문장 전체를 하나의 벡터로 변환하며, 우리가 사용한 ko-sroberta-multitask는 한국어 특화 모델(klue/roberta-base 기반)로 한국어 의미 이해 성능이 높다.

항목	내용
모델명	jhgan/ko-sroberta-multitask
기반 모델	klue/roberta-base (한국어 BERT 계열)
벡터 차원	768차원 (숫자 768개로 하나의 문장 표현)
정규화	normalize_embeddings=True → 단위벡터, 내적 = 코사인 유사도
처리 속도	batch_size=64, 13,335건 약 5-10분 소요

3-3 FAISS 인덱스 구축

왜 필요한가? 13,335개 벡터 중에서 사용자 입력과 가장 비슷한 것을 찾으려면 13,335번 유사도를 계산해야 한다. 건수가 늘어날수록 너무 느리다. FAISS(Facebook AI Similarity Search)는 대규모 벡터 검색을 밀리초 단위로 처리하는 라이브러리다.

인덱스 종류	IndexFlatIP (Inner Product = 코사인 유사도, 단위벡터 가정)
검색 방식	사용자 입력 텍스트 → SBERT 임베딩 → FAISS에서 top-k개 반환
저장 방식	faiss.serialize_index() → bytes → Path.write_bytes() (한글 경로 버그 우회)
산출물	embeddings.npy (13335×768), faiss.index, articles_meta.parquet

4-1 머신러닝 분류 문제로 정의

왜 필요한가? FAISS는 유사한 아티클을 찾아줄 뿐이다. '이 전략의 실패 확률'이라는 수치를 내려면 별도의 분류 모델이 필요하다. 임베딩 벡터(768차원)를 입력으로 받아 $P(\text{failure} | \text{임베딩}) = \text{실패 확률}$ 을 출력하는 모델을 학습한다.

학습 데이터: DBR + HBR + NAVER 합계 68,800건, neutral 제외하고 success(1)/failure(0)만 사용.

4-2 모델 비교 및 선택

4가지 모델을 5-fold Stratified Cross-Validation으로 비교했다:

모델	장점	ROC-AUC	F1(실패)	채택
Logistic Regression	빠르고 해석 용이	~0.89	~0.52	
LightGBM	트리 기반, 불균형 처리 강점	~0.96	~0.68	
XGBoost	LightGBM과 유사	~0.95	~0.65	
MLP (512→128)	비선형 패턴 포착력 최강	0.9771	0.78	채택

MLP를 선택한 이유 임베딩 벡터(768차원)는 고차원 비선형 패턴을 담고 있다. MLP(다층 퍼셉트론)는 은닉층을 통해 이런 비선형 패턴을 가장 잘 포착한다. hidden_layer_sizes=(512, 128) — 첫 층에서 특징 추출, 두 번째 층에서 정제.

4-3 3-factor 리스크 스코어 공식 (ai_server/main.py)

MLP 모델 점수만 쓰면 정보가 단순하다. 세 가지 신호를 조합해 더 정확한 리스크 스코어를 계산한다:

인수	비중	계산 방법	의미
model_score	35%	MLP 모델의 $P(\text{failure} \text{임베딩})$	전략 텍스트 자체의 실패 패턴 강도
case_score	45%	FAISS 유사 사례 중 failure 비율 (confidence 가중)	유사 사례들의 실제 실패 빈도
cluster_risk	20%	해당 클러스터의 사전 계산된 평균 실패율	산업/주제 군의 전반적 리스크

최종 공식:

```
base_score = 0.35 x model_score + 0.45 x case_score + 0.20 x cluster_risk
reliability = min(max_similarity / 0.65, 1.0)
risk_score = base_score x reliability + 0.5 x (1 - reliability)
```

reliability(신뢰도) 보정 이유 유사 사례가 전혀 없는 전략(max_similarity 낮음)은 불확실성이 크다. 이때 risk_score를 0.5(중립)쪽으로 수축시켜서 과신 예측을 방지한다.

5-1 UMAP 차원 축소

왜 필요한가? 768차원 벡터는 사람이 눈으로 볼 수 없다. 2차원으로 압축하면 '비슷한 주제의 아티클이 가까운 위치에 모이는 시맨틱 맵'을 그릴 수 있다. UMAP(Uniform Manifold Approximation and Projection)은 PCA보다 비선형 구조를 잘 보존하며 고차원 데이터의 군집 패턴을 시각화하는 데 뛰어나다.

파라미터	값	의미
n_components	2	출력 차원 (x, y 좌표)
n_neighbors	15	각 점이 고려하는 이웃 수. 높을수록 전역 구조 강조
min_dist	0.1	군집 내 점들이 얼마나 뭉칠지. 낮을수록 더 촘촘
metric	cosine	벡터 간 거리 기준 (임베딩과 동일하게 코사인)

5-2 K-means 클러스터링

왜 필요한가? UMAP 2D 좌표로 클러스터링하면 언어 차이(한국어/영어)에 의해 클러스터가 나뉘어버린다. 우리는 원래 768차원 임베딩으로 K-means를 돌려 주제 기반으로 클러스터를 나눴다. K=12로 설정한 결과 IT/AI, HR/조직, 마케팅, 금융, 글로벌 전략 등 의미 있는 비즈니스 주제 군집이 형성되었다.

cluster_id	주제 예시
1	IT / 디지털 전환 / AI
2	HR / 조직문화 / 리더십
3	마케팅 / 브랜드 / 고객
4	금융 / 투자 / M&A
5	글로벌 전략 / 해외시장
6	제조 / 공급망 / 물류
7	스타트업 / 혁신 / 창업
8-12	기타 (ESG, 규제·정책, R&D, 유통, 위기관리)

5-3 시맨틱 맵에서의 활용

각 아티클의 (umap_x, umap_y)와 cluster_id는 DB의 article_vectors 테이블에 저장된다. 프론트엔드 시맨틱 맵 화면에서 점으로 렌더링되며, 색상으로 성공/실패를 구분하고 영역별 클러스터 이름이 표시된다.

6. HBS 데이터 처리 시 주의사항

HBS(Harvard Business School) 데이터는 원문이 영어이다. 우리 파이프라인은 한국어 특화 모델(ko-sroberta)을 사용하므로 영어 원문을 그대로 넣으면 임베딩 품질이 크게 떨어진다.

단계	DBR / HBR 처리 방식	HBS 처리 방식
전처리	Kiwi 형태소 분석 (한국어)	content_ko_summary 컬럼 사용 (이미 번역 완료) Kiwi 형태소 분석 동일 적용 가능
임베딩	title + summary 한국어 텍스트	title_ko + content_ko_summary 사용 영어 원문 사용 금지
라벨링	동일 키워드 규칙 적용	한국어 번역 기반 동일 적용 가능
모델	동일 risk_model.pkl 사용	동일 모델 적용 (추가 학습 데이터로 활용 가능)

HBS_articles_k o.csv 컬럼 안내	title_ko: 제목 한국어 번역 / summary_ko: 요약 한국어 번역 / content_ko_summary: 본문 요약 한국어 번역 (2,116건 전체 완료)
---------------------------------------	--

7. 파이프라인 실행 순서 정리

환경 설정 `set PYTHONIOENCODING=utf-8 (Windows 필수) | conda activate [환경명] | pip install kiwipiepy sentence-transformers faiss-cpu umap-learn lightgbm xgboost`

순서	명령어	산출물	소요 시간
1	python 데이터처리/preprocess.py	DBR_preprocessed.parquet HBR_preprocessed.parquet	약 3-5분
2	python 데이터처리/embed.py	embeddings.npy faiss.index articles_meta.parquet	약 10-15분
3	python 데이터처리/label.py	DBR_labeled.parquet HBR_labeled.parquet	약 2-3분
4	python 데이터처리/risk_model.py	risk_model.pkl risk_model_report.txt	약 20-40분
5	python 데이터처리/umap_cluster_v3.py	umap_coords_v3.parquet cluster_info_v3.parquet	약 5분
6	python 데이터처리/import_articles_to_db.py	MySQL articles + article_labels 테이블	약 5-10분
7	python 데이터처리/import_umap_to_db_v3.py	MySQL article_vectors + clusters 테이블	약 3-5분

주의
(**embed.py**를 **label.py**보다 먼저 실행하는 이유)
label.py의 Stage 2(SBERT 재분류)에서 {DBR,HBR}_embeddings.npy 파일이 필요하다. embed.py가 먼저 실행되어 있어야 Stage 2가 동작한다. embed.py 없이 label.py를 실행하면 TF-IDF fallback으로 동작하므로 라벨 품질이 떨어진다.

요약: 각 단계의 핵심 목적

- ① 전처리 → '기계가 읽을 수 있는 형태'로 정제
- ② 라벨링 → '정답 데이터(성공/실패)' 자동 생성
- ③ 임베딩+FAISS → '의미 기반 검색 가능한 벡터 DB' 구축
- ④ 리스크 모델 → '실패 확률을 숫자로 예측'하는 AI 학습
- ⑤ UMAP+클러스터 → '사용자에게 시각적으로 보여줄' 지도 생성